

PRACTICAL COURSE WORKBOOK

Applied TypeScript

Build and test a responsive digital product

LEVEL	Intermediate
GUIDED TIME	5 hours across 12 lessons
PROJECT	a responsive, accessible application you can demonstrate
SKILLS	TypeScript, Generics, Utility Types, Critical evaluation, Documentation
EDITION	Structured draft - version 0.9

WHAT YOU WILL BE ABLE TO DO

- Use TypeScript appropriately in a realistic, bounded task.
- Use Generics appropriately in a realistic, bounded task.
- Use Utility Types appropriately in a realistic, bounded task.

WHO THIS IS FOR

Learners building accessible, maintainable web products.

BEFORE YOU BEGIN

- Comfort using a browser and text editor

Use this workbook with the online lessons. It is designed for A4 printing, handwritten evidence notes, and offline revision. It does not provide certification.

WORKBOOK GUIDE

A clear route through the course

Applied TypeScript is a structured, practical course covering TypeScript, Generics, Utility Types. It emphasizes explainable methods, checked work and a final outcome that can be reviewed.

MODULE	LESSONS AND FOCUS
01 TypeScript: Web foundations	1. Standards with TypeScript 2. Structure with Generics 3. Developer tools with Utility Types
02 Generics: Core implementation	1. Components with TypeScript 2. State and data with Generics 3. Responsive layout with Utility Types
03 Utility Types: Quality	1. Accessibility with TypeScript 2. Testing with Generics 3. Performance with Utility Types
04 TypeScript: Production practice	1. Security with TypeScript 2. Deployment with Generics 3. Maintenance with Utility Types

HOW TO USE EACH LESSON PAGE

- 1 Read the objective and identify the decision it supports.
- 2 Attempt the retrieval prompt before checking the online explanation.
- 3 Reproduce the worked example with the stated input.
- 4 Test one normal case and one boundary or failure case.
- 5 Record the result, limitation and next action in the evidence log.

PRINT AND FILE

Print at 100% scale on A4 paper. Use double-sided printing with long-edge binding. Keep completed pages with the project files named in the evidence log.

MODULE 01

TypeScript: Web foundations

Apply TypeScript through standards, structure, developer tools.

LESSON 01 / 18 MIN

Standards with TypeScript

Learning objectives

- Explain standards in the context of Applied TypeScript.
- Apply TypeScript to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

standards, TypeScript, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a standards result produced with TypeScript?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 02 / 22 MIN

Structure with Generics

Learning objectives

- Explain structure in the context of Applied TypeScript.
- Apply Generics to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

structure, Generics, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a structure result produced with Generics?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 03 / 26 MIN

Developer tools with Utility Types

Learning objectives

- Explain developer tools in the context of Applied TypeScript.
- Apply Utility Types to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

developer tools, Utility Types, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a developer tools result produced with Utility Types?

Evidence to keep

[] Original input [] Observed result [] Boundary or failure result [] Limitation and next action

MODULE REVIEW

Turn the module into usable evidence

1. RETRIEVE

Without checking the lessons, explain the module's three most important ideas in your own words.

2. APPLY

Describe the example or file you produced, the input used, and the result you observed.

3. REVIEW

Record one uncertainty, one source to recheck, and the next deliberate practice action.

MODULE 02

Generics: Core implementation

Apply Generics through components, state and data, responsive layout.

LESSON 04 / 30 MIN

Components with TypeScript

Learning objectives

- Explain components in the context of Applied TypeScript.
- Apply TypeScript to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

components, TypeScript, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a components result produced with TypeScript?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 05 / 18 MIN

State and data with Generics

Learning objectives

- Explain state and data in the context of Applied TypeScript.
- Apply Generics to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

state and data, Generics, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a state and data result produced with Generics?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 06 / 22 MIN

Responsive layout with Utility Types

Learning objectives

- Explain responsive layout in the context of Applied TypeScript.
- Apply Utility Types to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

responsive layout, Utility Types, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a responsive layout result produced with Utility Types?

Evidence to keep

[] Original input [] Observed result [] Boundary or failure result [] Limitation and next action

MODULE REVIEW

Turn the module into usable evidence

1. RETRIEVE

Without checking the lessons, explain the module's three most important ideas in your own words.

2. APPLY

Describe the example or file you produced, the input used, and the result you observed.

3. REVIEW

Record one uncertainty, one source to recheck, and the next deliberate practice action.

MODULE 03

Utility Types: Quality

Apply Utility Types through accessibility, testing, performance.

LESSON 07 / 26 MIN

Accessibility with TypeScript

Learning objectives

- Explain accessibility in the context of Applied TypeScript.
- Apply TypeScript to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

accessibility, TypeScript, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a accessibility result produced with TypeScript?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 08 / 30 MIN

Testing with Generics

Learning objectives

- Explain testing in the context of Applied TypeScript.
- Apply Generics to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

testing, Generics, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a testing result produced with Generics?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 09 / 18 MIN

Performance with Utility Types

Learning objectives

- Explain performance in the context of Applied TypeScript.
- Apply Utility Types to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

performance, Utility Types, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a performance result produced with Utility Types?

Evidence to keep

[] Original input [] Observed result [] Boundary or failure result [] Limitation and next action

MODULE REVIEW

Turn the module into usable evidence

1. RETRIEVE

Without checking the lessons, explain the module's three most important ideas in your own words.

2. APPLY

Describe the example or file you produced, the input used, and the result you observed.

3. REVIEW

Record one uncertainty, one source to recheck, and the next deliberate practice action.

MODULE 04

TypeScript: Production practice

Apply TypeScript through security, deployment, maintenance.

LESSON 10 / 22 MIN

Security with TypeScript

Learning objectives

- Explain security in the context of Applied TypeScript.
- Apply TypeScript to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

security, TypeScript, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a security result produced with TypeScript?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 11 / 26 MIN

Deployment with Generics

Learning objectives

- Explain deployment in the context of Applied TypeScript.
- Apply Generics to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

deployment, Generics, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a deployment result produced with Generics?

Evidence to keep

☐ Original input ☐ Observed result ☐ Boundary or failure result ☐ Limitation and next action

LESSON 12 / 30 MIN

Maintenance with Utility Types

Learning objectives

- Explain maintenance in the context of Applied TypeScript.
- Apply Utility Types to a bounded practical task.
- Evaluate the result using explicit quality criteria.

Key terms

maintenance, Utility Types, webdev

Retrieval prompt

In Applied TypeScript, which evidence best supports a maintenance result produced with Utility Types?

Evidence to keep

[] Original input [] Observed result [] Boundary or failure result [] Limitation and next action

MODULE REVIEW

Turn the module into usable evidence

1. RETRIEVE

Without checking the lessons, explain the module's three most important ideas in your own words.

2. APPLY

Describe the example or file you produced, the input used, and the result you observed.

3. REVIEW

Record one uncertainty, one source to recheck, and the next deliberate practice action.

CAPSTONE PROJECT

Produce a reviewable Applied TypeScript project using TypeScript, Generics, Utility Types.

DELIVERABLE	A working Applied TypeScript artefact plus an evidence-based self-review.
TOOLS	A suitable TypeScript environment, A plain-text decision log, Test data or realistic sample material

Production plan

- 1 Define the intended user, outcome and constraints.
- 2 Create the smallest complete result using TypeScript.
- 3 Test one normal case, one boundary case and one failure response.
- 4 Revise the work from the evidence and preserve before-and-after results.
- 5 Prepare a concise handover containing method, limitations and next step.

Definition of done

- The output matches the stated outcome.
- Inputs and decisions are reproducible.
- Boundary and failure evidence is included.
- Limitations and responsibility considerations are explicit.

PROJECT EVIDENCE

Document decisions another person can review

1. PURPOSE AND USER

State the intended user, need, expected outcome and constraints.

2. INPUTS AND ASSUMPTIONS

List source files, data, versions, permissions and assumptions.

PROJECT EVIDENCE

Tests, limitations and revision

3. NORMAL CASE

Expected result, observed result and evidence location.

4. BOUNDARY OR FAILURE CASE

What was difficult, what happened and why it matters.

5. REVISION AND LIMITATION

What changed, what improved and what remains unproven.

REVIEW AND SOURCES

Make the work credible and repeatable

Final self-review

- Can another learner repeat the method?
- Did I test a difficult case?
- Did I avoid unsupported claims?
- Is the next action proportionate to the remaining risk?

FINAL DECISION

Is the project ready to share? State the evidence, remaining risk, owner and next review date.

Authoritative references

Learn web development

MDN Web Docs - reviewed 2026-08-21

https://developer.mozilla.org/en-US/docs/Learn_web_development

WCAG 2.2

W3C - reviewed 2026-08-21

<https://www.w3.org/TR/WCAG22/>

SOURCE NOTE

Product versions, laws and professional standards can change. Recheck the linked primary source before applying version-sensitive information.

NEXT IMPROVEMENT

Choose one weakness found during review and improve it before extending the TypeScript scope.